

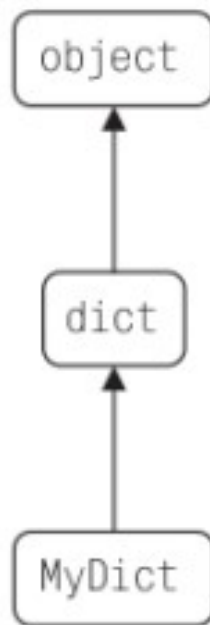
Python

12

Суперклас для класів dict та Mydict
Базовий клас для класів dict та Mydict

Суперклас для класса Mydict
Базовый класс для класса Mydict

Подкласс класса dict
Специализация класса dict
Дочерний класс класса dict



Подкласс класса object
Специализация класса object
Дочерний класс класса object

Подкласс класса object
Специализация класса object
Дочерний класс класса object

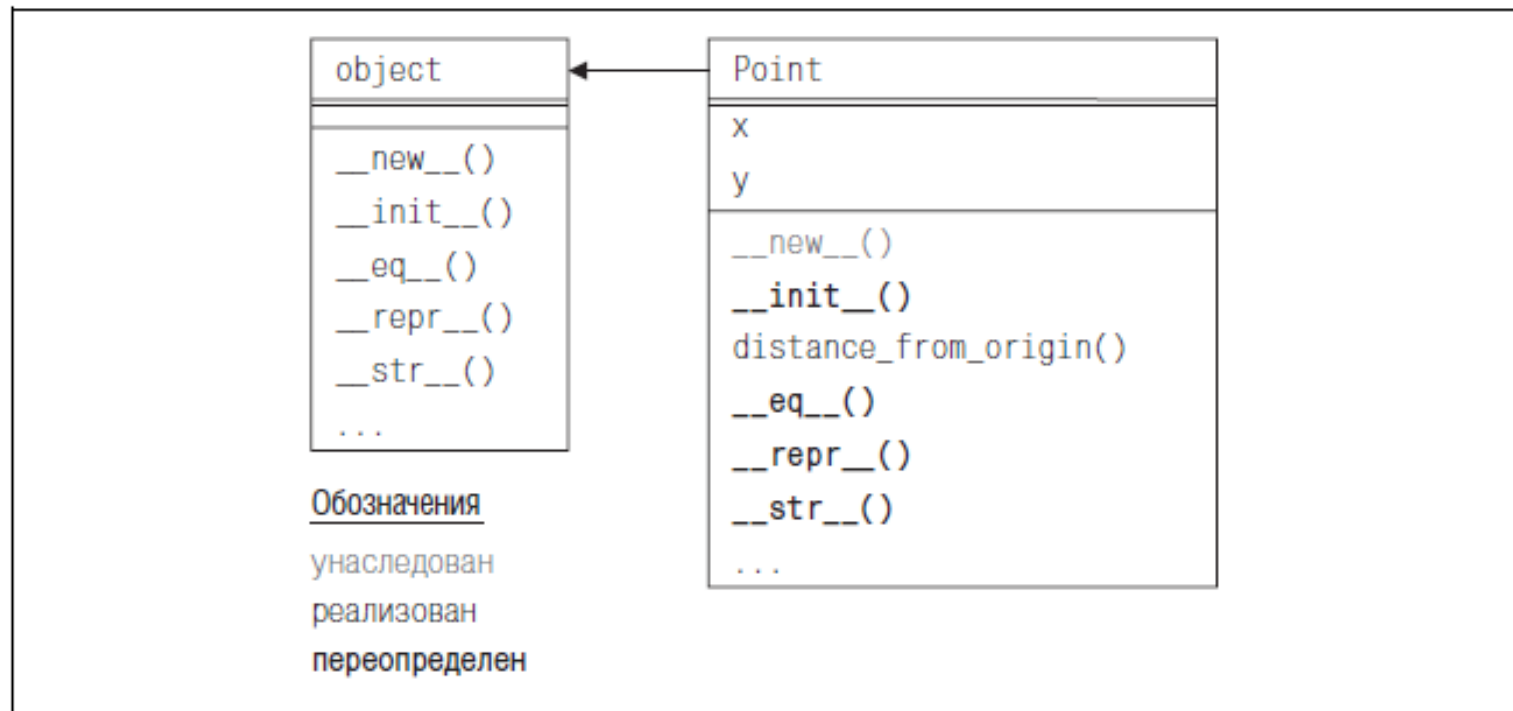
Спеціальні методи порівняння

<code>__lt__(self, other)</code>	<code>x < y</code>	Повертає True , якщо x менше за y
<code>__le__(self, other)</code>	<code>x <= y</code>	Повертає True , якщо x менше або доівнює y
<code>__eq__(self, other)</code>	<code>x == y</code>	Повертає True , якщо x доівнює y
<code>__ne__(self, other)</code>	<code>x != y</code>	Повертає True , якщо x не доівнює y
<code>__ge__(self, other)</code>	<code>x >= y</code>	Повертає True , якщо x більше або доівнює y
<code>__gt__(self, other)</code>	<code>x > y</code>	Повертає True , якщо x більше за y

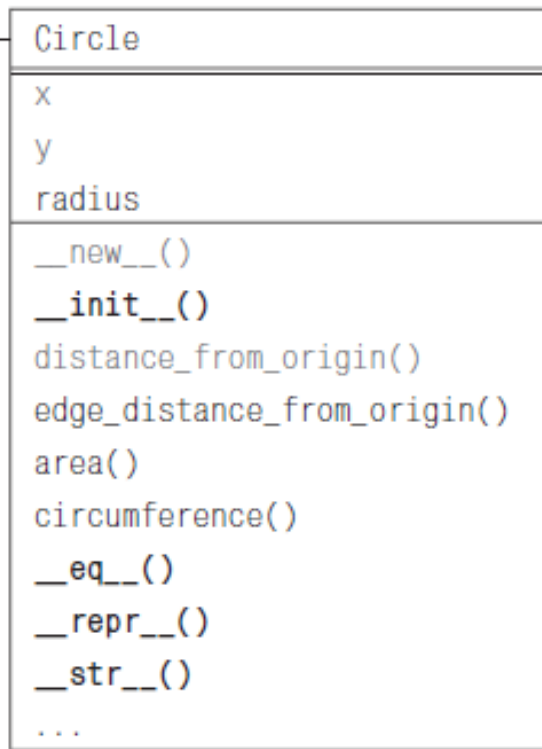
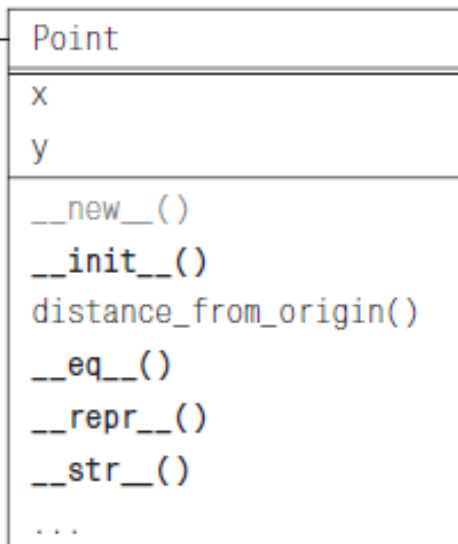
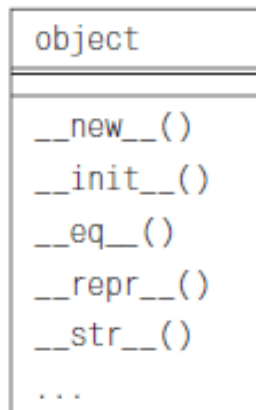
Деякі спеціальні арифметичні методи

Специальный метод	Пример использования	Специальный метод	Пример использования
<code>__abs__(self)</code>	<code>abs(x)</code>	<code>__complex__(self)</code>	<code>complex(x)</code>
<code>__float__(self)</code>	<code>float(x)</code>	<code>__int__(self)</code>	<code>int(x)</code>
<code>__index__(self)</code>	<code>bin(x)</code> <code>oct(x)</code> <code>hex(x)</code>	<code>__round__(self, <i>digits</i>)</code>	<code>round(x, <i>digits</i>)</code>
<code>__pos__(self)</code>	<code>+x</code>	<code>__neg__(self)</code>	<code>-x</code>
<code>__add__(self, other)</code>	<code>x + y</code>	<code>__sub__(self, other)</code>	<code>x - y</code>
<code>__iadd__(self, other)</code>	<code>x += y</code>	<code>__isub__(self, other)</code>	<code>x -= y</code>
<code>__radd__(self, other)</code>	<code>y + x</code>	<code>__rsub__(self, other)</code>	<code>y - x</code>
<code>__mul__(self, other)</code>	<code>x * y</code>	<code>__mod__(self, other)</code>	<code>x % y</code>
<code>__imul__(self, other)</code>	<code>x *= y</code>	<code>__imod__(self, other)</code>	<code>x %= y</code>
<code>__rmul__(self, other)</code>	<code>y * x</code>	<code>__rmod__(self, other)</code>	<code>y % x</code>
<code>__floordiv__(self, other)</code>	<code>x // y</code>	<code>__truediv__(self, other)</code>	<code>x / y</code>
<code>__ifloordiv__(self, other)</code>	<code>x //= y</code>	<code>__itruediv__(self, other)</code>	<code>x /= y</code>
<code>__rfloordiv__(self, other)</code>	<code>y // x</code>	<code>__rtruediv__(self, other)</code>	<code>y / x</code>

Дерево успадкування класу Point



Дерево усядкування класу Circle



Обозначения

унаследован

реализован

переопределен

MRO

```
1 class O: ...
2 class A(O): ...
3 class B(O): ...
4 class C(O): ...
5 class D(O): ...
6 class E(O): ...
7 class K1(A, B, C): ...
8 class K2(B, D): ...
9 class K3(C, D, E): ...
10 class Z(K1, K2, K3): ...
```

Спробуйте відразу назвати порядок пошуку способів у цій ієрархії

У цьому допомагає метод `mro()`

Навіщо потрібен @staticmethod

```
1 class Example:
2     def hello():
3         print('hello')
4
5     def hello_2(self):
6         print('hello_2')
```

Можна викликати метод **hello** від класу, але не вийде викликати від екземпляра. І навпаки, у разі виклику методу **hello_2** можна викликати його від екземпляра, але не можна від класу.

Статикметод вирішує цю проблему:

```
1 class Example:
2     @staticmethod
3     def hello():
4         print('hello')
```

```
1 Example.hello()
2 hello
3
4 p = Example()
5 p.hello()
6 hello
```


Навіщо потрібен **@staticmethod**

@staticmethod — використовується для створення методу, який нічого не знає про клас або екземпляр, через який він був викликаний. Він просто отримує передані аргументи, без неявного першого аргументу, і його визначення незмінне через спадкування.

Простіше кажучи, **@staticmethod** - це начебто звичайної функції, визначеної всередині класу, яка не має доступу до екземпляра, тому її можна викликати без створення екземпляра класу.

Навіщо потрібен @classmethod

```
class Example:
    @classmethod
    def class_hello(cls):
        return f'class_hello {cls}'
Example.class_hello()
"class_hello <class '__main__.Example'>"

p = Example()
p.class_hello()
"class_hello <class '__main__.Example'>"
```

Навіщо потрібен **@classmethod**

@classmethod - це метод, який отримує клас як неявний перший аргумент, так само, як звичайний метод екземпляра отримує екземпляр. Це означає, що ви можете використовувати клас та його властивості всередині цього методу, а не конкретного екземпляра.

Простіше кажучи, **@classmethod** - це звичайний метод класу, що має доступ до всіх атрибутів класу, через який він був викликаний. Отже, `classmethod` - це метод, який прив'язаний до класу, а не до екземпляра класу.

Навіщо потрібен @property

```
1 class Money:
2     def __init__(self, name, money):
3         self.name = name
4         self.money = money
5     @property
6     def get_balance(self):
7         return self.money
```

```
1 bob = Money('Bob', 1000)
2 bob.get_balance
3 1000
```

Навіщо потрібен **@property**

@property декоратор може бути використаний для визначення методів у класі, які діють як атрибути.

Щоб мати можливість присвоїти значення методу, як атрибуту - потрібно використовувати сетер (**setter**).

Щоб мати можливість створити сеттер, у вас має бути визначений гетер (**@property**)

Deleter - цей метод викликається тоді, коли відбувається видалення значення властивості (метода). **Deleter** дозволяє насамкінець зробити щось необхідне, наприклад, закрити відкритий раніше файл або мережеве з'єднання, словом, "прибрати за собою".

Щоб мати можливість створити делетер, у вас має бути визначений гетер (**@property**)

Навіщо потрібен `@dataclass`

Клас даних - це клас, який (в основному) зберігає дані, хоча насправді немає жодних обмежень.

Він розроблений за допомогою декоратора `@dataclass`.

Класи даних - це звичайні класи, орієнтовані на збереження стану та не містять багато логіки.

Кожного разу, коли ви створюєте клас, який складається в основному з атрибутів, ви можете створювати клас даних.

Модуль **`dataclasses`** спрощує створення класів даних.